

Lab 1 — Linux Kernel Module and Process Representation in Linux

Operating Systems, Comp Sci 3SH3, Fall 2026

Prof. Neerja Mhaskar

Lab Format

1. You must show your working solution of this lab to the TA for a grade.
2. Working directly on your local Linux machine, instead of a virtual machine (VM), may lead to kernel panic if mistakes are made. This could require you to reinstall your operating system and possibly lose data. Therefore, it is advisable to work on your VM for this lab.
3. The TA will check your solution and will quiz you on your work. After which they will enter your mark and feedback on Avenue.
4. If you do not show your work to your Lab TA, you will get a zero (unless you provide an MSAF, in which case you will get a 5-day extension to get this lab graded).
5. It is your responsibility to connect with your Lab TA to get a grade and ensure that your grade has indeed been posted on Avenue.

Outline

In this lab you will learn the following:

1. How to create a kernel module and load/remove them from the Linux kernel.
2. How processes are represented in Linux and how to access members of the idle/swapper process.

This lab should be completed using the Linux virtual machine (see Take Home Lab for installation instructions). Although you may use any editor to write these C programs, you will have to use the terminal application (on the virtual machine) to compile the programs, and you will have to enter commands on the command line to manage the modules in the kernel. As you'll discover, the advantage of developing kernel modules is that it is a relatively easy method of interacting with the kernel, thus allowing you to write programs that directly invoke kernel functions. It is important for you to keep in mind that you are indeed writing kernel code that directly interacts with the kernel. That normally means that any errors in the code could crash the system! However, since you will be using a virtual machine, any failures will at worst only require rebooting the system.

Part 1 — Linux Kernel Modules

You can list all kernel modules that are currently loaded by entering the command

```
lsmod
```

This command will list the current kernel modules in three columns: name, size, and where the module is being used.

The program `simple.c` posted on the course website under **Content** → **Lab** → **Lab 1** illustrates a very basic kernel module that prints appropriate messages when it is loaded and unloaded.

The function `simple_init()` is the module entry point, which represents the function that is invoked when the module is loaded into the kernel. Similarly, the `simple_exit()` function is the module exit point—the function that is called when the module is removed from the kernel.

The module entry point function must return an integer value, with 0 representing success and any other value representing failure. The module exit point function returns `void`. Neither the module entry point nor the module exit point is passed any parameters. The two following macros are used for registering the module entry and exit points with the kernel:

```
module_init(simple_init)
module_exit(simple_exit)
```

Notice how the module entry and exit point functions make calls to the `printk()` function. `printk()` is the kernel equivalent of `printf()`, but its output is sent to a kernel log buffer whose contents can be read by the `dmesg` command. One difference between `printf()` and `printk()` is that `printk()` allows us to specify a priority flag, whose values are given in the `<linux/printk.h>` include file. In this instance, the priority is `KERN_INFO`, which is defined as an informational message.

The final lines—`MODULE_LICENSE()`, `MODULE_DESCRIPTION()`, and `MODULE_AUTHOR()`—represent details regarding the software license, description of the module, and author. For our purposes, we do not require this information, but we include it because it is standard practice in developing kernel modules.

This kernel module `simple.c` is compiled using the `Makefile` accompanying the source code is also posted under **Content** → **Labs** → **Lab1**. To compile the module, enter the following on the command line:

```
make
```

You may need to install some packages before using the `make` command, depending on the error you may face:

```
sudo apt install make
sudo apt install gcc
sudo apt install gcc-12          # or a newer gcc if your kernel requires it
sudo apt install --reinstall linux-headers-$(uname -r)
```

Tip

To avoid compiler errors and warnings, it is best to compile your kernel module with the same major GCC version used to build the running kernel. Check the kernel's compiler with `cat /proc/version`. If it reports a newer GCC version (e.g., `gcc-13` or `gcc-14`), install and use that version instead of `gcc-12`.

You need to run the `make` command in the directory which contains the `Makefile`.

The compilation produces several files. The file `simple.ko` represents the compiled kernel module. The following step illustrates inserting this module into the Linux kernel.

Loading and Removing Kernel Modules

Kernel modules are loaded using the `insmod` command, which is run as follows:

```
sudo insmod simple.ko
```

To check whether the module has loaded, enter the `lsmod` command and search for the module `simple`. Recall that the module entry point is invoked when the module is inserted into the kernel. To check the contents of this message in the kernel log buffer, enter the command

```
dmesg
```

You should see the message “Loading Module.”

Removing the kernel module involves invoking the `rmmmod` command (notice that the `.ko` suffix is unnecessary):

```
sudo rmmmod simple
```

Be sure to check with the `dmesg` command to ensure the module has been removed.

Because the kernel log buffer can fill up quickly, it often makes sense to clear the buffer periodically. This can be accomplished as follows:

```
sudo dmesg -c
```

Proceed through the steps described above to create the kernel module and to load and unload the module. Be sure to check the contents of the kernel log buffer using `dmesg` to ensure that you have followed the steps properly.

Part 2 — Process Representation in Linux

You can use the file (`simple.c`) you created/used under Part 1 of this lab to complete Part 2.

Processes in Linux are represented by the `task_struct` data structure as shown in the Chapter 3 lecture slides. This data structure is defined in the `sched.h` header file. Take a look at the definition of this data structure and study the following members of this structure: `pid`, `state`, `flags`, `rt_priority` (runtime priority), `policy` (process policy) and `tgid` (task group id).

In this part of the lab, you will write a Linux kernel module which outputs the values of the above-listed members to the kernel log buffer for the init task (also called the swapper/idle process) in a Linux system, when the module is loaded into the kernel.

Print the output of the `dmesg` command to a file and show it to your TA. (After modifying the C file, you need to compile it and load the kernel module again: `make`, `insmod`, and then `dmesg`.)

Kernel version note — the “state” field

Since Linux 5.14, the process state field in `task_struct` was renamed from `state` to `__state` and its type is `unsigned int`. If your VM runs a recent kernel and the code does not compile because `state` is undeclared, use `task->__state` and print it with the `%u` format specifier:

```
printk(KERN_INFO "init_task state:%u\n", task->__state);
```

On older kernels (before 5.14) keep `task->state` printed with `%ld`. You can check your kernel version with `uname -r`.

Note

The `task_struct` data structure for the Linux swapper (idle process) is named `init_task` and it has a `pid = 0`. It is a task scheduled to run whenever no other runnable task is available on the CPU.

If you are using the `simple.c` file, you need only write a procedure `print_init_PCB()` that prints the required members of `init_task`. Then call this function in `simple_init()`.

Sample Output

```
Loading Module
[ 2300.008729] init_task pid:0
[ 2300.008733] init_task state:0
[ 2300.008736] init_task flags:2097152
[ 2300.008740] init_task runtime priority:0
[ 2300.008743] init_task process policy:0
[ 2300.008745] init_task tgid:0
```

End of Lab 1